

ASR Benchmark Report for Indian Conversational Speech

1. Executive Summary

This project benchmarks Automatic Speech Recognition (ASR) systems for Indian conversational speech in a telephony-style environment. The evaluation focuses on Bangalore locality names spoken naturally in Hindi, Hinglish, Tanglish, and mixed accent conversational speech.

The primary goal was not perfect transcription, but whether the ASR system could correctly capture locality entities important for a blue-collar hiring platform.

Three ASR systems were evaluated:

- Deepgram Nova-2 (baseline)
- OpenAI Whisper
- Sarvam Saarika v2.5

Final Recommendation:

Based on the experiments, **Sarvam Saarika v2.5** performed best for Indian conversational speech and locality extraction.

Model	Locality Detection Rate	Avg Latency
Deepgram Nova-2	35%	2.10s
Whisper	65%	2.20s
Sarvam Saarika v2.5	70%	1.95s

Sarvam achieved the highest locality detection rate while also maintaining the lowest latency. Whisper was a strong fallback option due to its robustness and open-source flexibility. Deepgram struggled significantly with code switching and accented Indian speech.

2. Dataset & Recording Methodology

A custom dataset of 20 recordings was created using a mobile phone microphone.

Each recording:

- Included Bangalore locality names naturally inside sentences
- Used conversational speech instead of scripted pronunciation
- Mixed Hindi, English, Hinglish, and Tanglish
- Included quiet and noisy conditions

Examples:

- “Main JP Nagar side rehta hoon.”
- “Koramangala mein traffic bahut zyada hai.”
- “Naan Bellandur la oru dress shop pakkathula irukken.”

The goal was to simulate realistic user audio from a hiring platform instead of clean studio speech. The 20 recordings included variation in: quiet vs noisy environments, normal vs rushed speech, Hindi/Hinglish/Tanglish, and mobile recording.

3. Methodology & Rationale

Standard ASR benchmarks use Word Error Rate (WER). However, requiring a perfect human reference transcript for every WhatsApp audio note or phone call is unscalable. Furthermore, standard WER penalizes acceptable code-switching (e.g., if a user says *"Main Peenya mein hoon"* and the model outputs *"I am in Peenya"*, standard WER scores it as a failure, even though the business outcome is achieved.

To solve this, I designed a **Reference-Free Evaluation Pipeline**. This evaluation is intended as a practical stress-test rather than a statistically rigorous benchmark.

- 1. **Locality Detection Rate (LDR):** The primary metric. Using a dictionary of canonical Bangalore localities and their phonetic variants, I used fuzzy matching to check if the core entity was extracted, regardless of the surrounding grammar or language.
- 2. **Cross-Model Agreement Rate (CMAR):** Used RapidFuzz to measure pairwise similarity across model outputs. High agreement serves as a proxy for audio clarity when ground truth is missing.
- 3. **Transcript Entropy:** Measured word-level chaos across models. Higher entropy indicated severe acoustic ambiguity or language mismatch.
- 4. **Hallucination Risk:** A heuristic metric designed to identify fabricated or unrelated outputs. Sarvam produced the lowest hallucination risk.

Model	Hallucination Risk
Deepgram	0.07
Whisper	0.07
Sarvam	0.05

4. Surprising Insights

1, The Transliteration Trap: Sarvam outputs native Devanagari script (e.g., "केंगेरी"), while Whisper outputs Romanized English ("Kengeri"). Initially, this caused my evaluation script to score Sarvam at 0% because standard string matching failed across scripts. I had to implement a transliteration layer (unidecode) to normalize the text before calculating Cross-Model Agreement.

2, Deepgram's Code-Switching Collapse: Deepgram (nova-2) severely struggles with thick accents and code-switching. Instead of hallucinating, it simply drops the audio it doesn't understand.

- *Example (Peenya.wav):* The user spoke in heavily accented Hinglish. Whisper extracted "Penia", Sarvam extracted "पेनिया", but Deepgram simply returned: "No."
- *Example (Bellandur.wav):* User spoke in Tanglish. Whisper and Sarvam perfectly extracted Bellandur. Deepgram returned: "You know, dress up."

5. Failure Analysis

Even the best models completely failed on 4 out of 20 files. Analyzing these reveals three major blind spots in current ASRs:

1. **Severe Phonetic Drift in Fast Speech:** In Koramangala.wav, the user spoke very quickly. Whisper heard "*Kuromakulam*" and Sarvam heard "*क्रोमा कलम*" (Kroma Kalam). The phonetic drift was too wide for even an 80% fuzzy-match threshold to catch.
2. **Colloquial Acronyms vs. Canonical Names:** The assignment provided "Rajarajeshwarinagar", but the speaker natively called it "RR Nagar". Whisper hallucinated this as "*Haran Nagar*", and Sarvam heard "*आरा नगर*" (Ara Nagar). We must map local acronyms (RR Nagar, JP Nagar) to canonical names in our production entity-extraction logic, as models fail to bridge this gap acoustically.
3. **The Migrant Worker Pronunciation Gap (Geographic Accent Mismatch):** Many people are migrant workers from other states who are completely unfamiliar with native Bangalore pronunciations. When a North Indian candidate with a strong regional accent attempts to say a complex Dravidian locality (e.g., *Byatarayanapura* or *Thalaghattapura*), the resulting pronunciation is heavily distorted. The ASR models fail to map this "accent-layered mispronunciation" to the correct text, proving that aggressive fuzzy-matching is an absolute necessity for our specific user base, not just a nice-to-have.

6. Limitations

1. **Sample Size & Geographic Bias:** This evaluation was run on 20 pristine audio samples limited to Bangalore localities. Performance on North Indian or Tier-3 city localities is unknown.
2. **Hardware vs. API Latency:** Whisper was run locally, while Deepgram/Sarvam relied on API calls. In production, hosting our own Whisper instances would completely eliminate the API data privacy risk and network latency, though it requires GPU overhead.
3. **Fuzzy Thresholds:** The LDR metric uses an 80% fuzzy match threshold. In production, this threshold will require fine-tuning to prevent false positives between phonetically similar localities (e.g., *Bellandur* vs *Bennigana Halli*).